

PATRONES

COMPOSITE

Patrón estructural cuyo objetivo es tratar de manera uniforme objetos individuales y composiciones de objetos. Se usa cuando se quiere representar jerarquías de tipo árbol, donde algunos nodos son elementos simples (hojas) y otros son contenedores (composite) que pueden tener hijos.

Los elementos que son compuestos tienen la capacidad de calcular su propio precio (es decir, pueden calcular el precio de cada uno de sus elementos)

```
// Component
interface ItemEnvio {
    double getPrecio();
}

// Leaf
class Producto implements ItemEnvio {
    private String nombre;
    private double precio;

    public Producto(String nombre, double precio) {
        this.nombre = nombre;
        this.precio = precio;
    }

    @Override
    public double getPrecio() {
        return precio;
    }
}

// Nuevo Leaf: ComboPromocional
class ComboPromocional implements ItemEnvio {
    private String descripcion;
    private double precioCombo;

    public ComboPromocional(String descripcion, double precioCombo) {
        this.descripcion = descripcion;
        this.precioCombo = precioCombo;
    }

    @Override
    public double getPrecio() {
        DayOfWeek hoy = LocalDate.now().getDayOfWeek();
        if (hoy == DayOfWeek.MONDAY) {
            return precioCombo * 0.5; // 50% de descuento los lunes
        }
        return precioCombo;
    }
}
```

```

// Composite
class Caja implements ItemEnvio {
    private List<ItemEnvio> contenidos = new ArrayList<>();

    public Caja(ItemEnvio... items) {
        contenidos.addAll(Arrays.asList(items));
    }

    @Override
    public double getPrecio() {
        double total = 0;
        for (ItemEnvio item : contenidos) {
            total += item.getPrecio();
        }
        return total;
    }
}

// Cliente
public class MercadoLibreComposite {
    public static void main(String[] args) {
        ItemEnvio almohada = new Producto("Almohada", 20);
        ItemEnvio celular = new Producto("Celular", 1100);
        ItemEnvio destornillador = new Producto("Destornillador", 13);
        ItemEnvio notebook = new Producto("Notebook", 2200);

        ItemEnvio comboOficina = new ComboPromocional("Combo Oficina", 200); // Agrego el nuevo combo

        ItemEnvio cajaPequena1 = new Caja(celular, comboOficina); // Lo agrego en una caja sin modificar nada
        ItemEnvio cajaPequena2 = new Caja(destornillador);

        ItemEnvio cajaMediana1 = new Caja(cajaPequena1, cajaPequena2);
        ItemEnvio cajaMediana2 = new Caja(notebook);

        ItemEnvio cajaGrande = new Caja(almohada, cajaMediana1, cajaMediana2);

        System.out.println("Composite ----> Precio total del envío: $" + cajaGrande.getPrecio());
    }
}

```

Otro ejemplo:

Copiar código

```
// COMPONENT
interface FileSystemNode {
    void mostrar(int nivel);
}

// LEAF
class Archivo implements FileSystemNode {
    private String nombre;

    public Archivo(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void mostrar(int nivel) {
        System.out.println("  ".repeat(nivel) + "- " + nombre);
    }
}

// COMPOSITE
class Carpeta implements FileSystemNode {
    private String nombre;
    private List<FileSystemNode> hijos = new ArrayList<>();

    public Carpeta(String nombre) {
        this.nombre = nombre;
    }

    public void agregar(FileSystemNode nodo) {
        hijos.add(nodo);
    }

    @Override
    public void mostrar(int nivel) {
        System.out.println("  ".repeat(nivel) + "+ " + nombre);
        for (FileSystemNode hijo : hijos) {
            hijo.mostrar(nivel + 1);
        }
    }
}

// CLIENTE
public class Main {
    public static void main(String[] args) {
        Carpeta root = new Carpeta("root");
        Carpeta fotos = new Carpeta("fotos");
        Carpeta musica = new Carpeta("musica");

        Archivo a1 = new Archivo("perfil.png");
        Archivo a2 = new Archivo("vacaciones.jpg");
        Archivo m1 = new Archivo("rock.mp3");

        fotos.agregar(a1);
        fotos.agregar(a2);
        musica.agregar(m1);

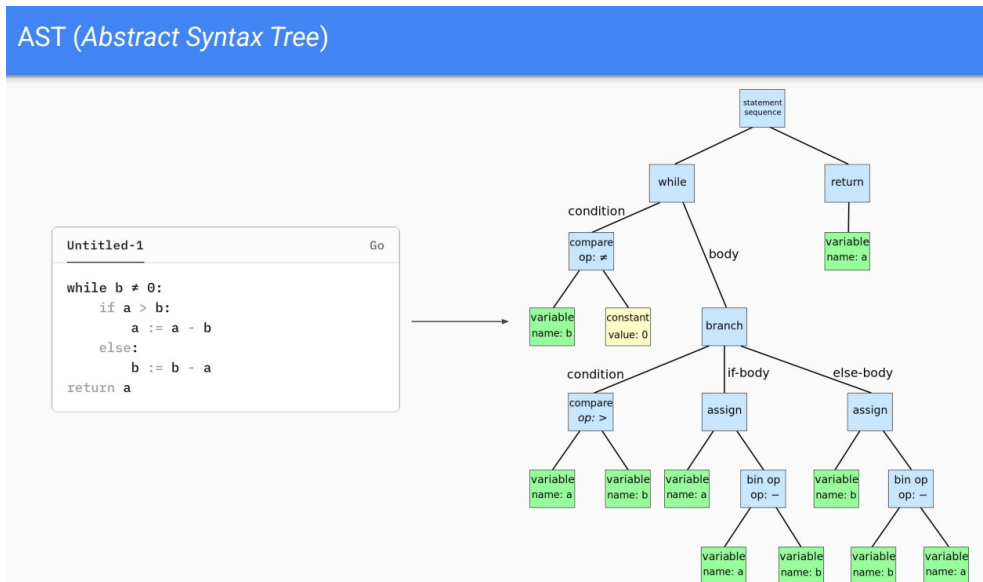
        root.agregar(fotos);
        root.agregar(musica);

        root.mostrar(0);
    }
}
```

↓

INTERPRETER

Patrón usado cuando se tiene un lenguaje, usualmente pequeño y específico al dominio, y se quiere evaluar las oraciones escritas en ese lenguaje. Se usa cuando se desea interpretar un lenguaje simple, acotado y su gramática cambia poco, y cuando nos beneficiamos de componer sus reglas



Gramática BNF

```
expression ::= condition | expression AND expression | expression OR expression
condition ::= category_is CATEGORY | price_less_than NUMBER | in_stock
```

Ejemplo:

```
1 Interfaz base: Expression

java
interface Expression {
    boolean interpretar(Contexto ctx);
}

2 Terminal Expression: Variable booleana

java
class Variable implements Expression {
    private String nombre;

    public Variable(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public boolean interpretar(Contexto ctx) {
        return ctx.getValor(nombre);
    }
}
```

3 Contexto que almacena los valores de cada variable

```
java Copiar código

import java.util.Map;

class Contexto {
    private Map<String, Boolean> valores;

    public Contexto(Map<String, Boolean> valores) {
        this.valores = valores;
    }

    public boolean getValor(String nombre) {
        return valores.getOrDefault(nombre, false);
    }
}
```

4 Non-Terminal Expressions (AND, OR, NOT)

AND

```
java Copiar código

class AndExpression implements Expression {
    private Expression izquierda;
    private Expression derecha;

    public AndExpression(Expression izquierda, Expression derecha) {
        this.izquierda = izquierda;
        this.derecha = derecha;
    }

    @Override
    public boolean interpretar(Contexto ctx) {
        return izquierda.interpretar(ctx) && derecha.interpretar(ctx);
    }
}
```

OR

```
java Copiar código

class OrExpression implements Expression {
    private Expression izquierda;
    private Expression derecha;

    public OrExpression(Expression izquierda, Expression derecha) {
        this.izquierda = izquierda;
        this.derecha = derecha;
    }

    @Override
    public boolean interpretar(Contexto ctx) {
        return izquierda.interpretar(ctx) || derecha.interpretar(ctx);
    }
}
```

NOT

```
java Copiar código

class NotExpression implements Expression {
    private Expression expr;

    public NotExpression(Expression expr) {
        this.expr = expr;
    }

    @Override
    public boolean interpretar(Contexto ctx) {
        return !expr.interpretar(ctx);
    }
}
```

(Juan AND Pedro) OR (NOT Maria)

```
Código:
java Copiar código

import java.util.Map;

public class Main {
    public static void main(String[] args) {

        // Valores asignados a las variables
        Contexto ctx = new Contexto(Map.of(
            "Juan", true,
            "Pedro", false,
            "Maria", true
        ));

        // Expresión: (Juan AND Pedro) OR (NOT Maria)
        Expression expression = new OrExpression(
            new AndExpression(
                new Variable("Juan"),
                new Variable("Pedro")
            ),
            new NotExpression(
                new Variable("Maria")
            )
        );

        boolean resultado = expression.interpretar(ctx);
        System.out.println("Resultado = " + resultado);
    }
}
```

Ventajas: modularidad, extensibilidad (Open-Closed), Readability (el código sigue la gramática).

Desventajas: aplicabilidad limitada, hay alternativas más performantes.

OBSERVER

El patrón observer define una relación uno a muchos: cuando un objeto (Subject) cambia su estado, notifica automáticamente a todos sus observadores (Observers). Se usa cuando: se tienen múltiples componentes que deben actualizarse cuando algo cambia; se quiere bajo acoplamiento (el Subject no sabe detalles de quién lo observa).

Sin este patrón, cada parte del sistema (los Observers) tendrían que pedir actualizaciones por su cuenta, produciendo *Polling* repetido, alto acoplamiento y una dificultad para agregar nuevos observadores más adelante. Así, Observer elimina el *polling* y mantiene los componentes independientes

Ejemplo:

1. La interfaz Observer

java

 Copiar código

```
interface Observer {  
    void update(float temperatura);  
}
```

2. La interfaz Subject

java

 Copiar código

```
interface Subject {  
    void agregarObserver(Observer o);  
    void eliminarObserver(Observer o);  
    void notificarObservers();  
}
```

3. Implementación del Subject

java

 Copiar código

```
import java.util.ArrayList;  
import java.util.List;  
  
class WeatherStation implements Subject {  
    private List<Observer> observers = new ArrayList<>();  
    private float temperatura;  
  
    @Override  
    public void agregarObserver(Observer o) {  
        observers.add(o);  
    }  
  
    @Override  
    public void eliminarObserver(Observer o) {  
        observers.remove(o);  
    }  
  
    @Override  
    public void notificarObservers() {  
        for (Observer o : observers) {  
            o.update(temperatura);  
        }  
    }  
  
    // Cambia la temperatura y notifica  
    public void setTemperatura(float nuevaTemp) {  
        this.temperatura = nuevaTemp;  
        notificarObservers();  
    }  
}
```

4. Observers concretos

```
java Copiar código

class Display implements Observer {
    private String nombre;

    public Display(String nombre) {
        this.nombre = nombre;
    }

    @Override
    public void update(float temperatura) {
        System.out.println(nombre + " muestra temperatura: " + temperatura);
    }
}
```

5. Uso del patrón

```
java Copiar código

public class Main {
    public static void main(String[] args) {
        WeatherStation estacion = new WeatherStation();

        Observer display1 = new Display("Display pequeño");
        Observer display2 = new Display("Display gigante");

        estacion.agregarObserver(display1);
        estacion.agregarObserver(display2);

        estacion.setTemperatura(25.3f);
        estacion.setTemperatura(30.0f);
    }
}
```

La lista de Observers del Subject es dinámica, por lo que estos pueden unirse o abandonarla cuando lo deseen

Hay dos estilos de hacer la notificación (el update):

- **Push:** al momento de hacer el update, esta se hace de una, notificando a cada uno de los observers.
- **Pull:** lo que se hace es que se manda algo a los observers, y estos reaccionan trayéndose la información en vez de recibirla directamente.

● PUSH (el Subject envía los datos)

✓ Ventajas

1. Menos llamadas: los datos vienen en la notificación, no hay que pedirlos luego.
2. Más eficiente cuando hay muchos observers o cuando la información es costosa de calcular.
3. Implementación simple: el Observer recibe exactamente lo que necesita.
4. Ideal para datos pequeños o bien definidos (ej.: temperatura, precio).

✗ Desventajas

1. Mayor acoplamiento: el Subject debe saber qué datos necesitan los observers.
2. Menos flexible: si los observers necesitan nueva info → hay que modificar el método `update()`.
3. Observadores "obligados" a recibir info que tal vez no usan.
4. Problema de "sobrecarga": si se envía mucha información, se desperdicia.

● PULL (el Observer pide los datos al Subject)

✓ Ventajas

1. Bajo acoplamiento: el Subject NO sabe qué datos necesita cada observer.
2. Alta flexibilidad: podés agregar nuevos observers sin tocar el Subject.
3. Observers personalizan qué datos leer: solo acceden a lo que necesitan.
4. Fácil de mantener y extender (ideal en patrones como MVC).

✗ Desventajas

1. Más llamadas y overhead:
Cada observer debe consultar al Subject (a veces varias getters).
2. Riesgo de leer datos inconsistentes:
Si el estado cambia entre la notificación y el pull.
3. Implementación más verbosa:
Cada observer debe conocer al Subject para consultarlo.
4. Menos eficiente si los observers son muchos o si los getters son costosos.

Ejemplo con pull:

1. Interfaz Observer (sin parámetros)

```
java Copiar código

interface Observer {
    void update();
}
```

2. Interfaz Subject

```
java Copiar código

interface Subject {
    void agregarObserver(Observer o);
    void eliminarObserver(Observer o);
    void notificarObservers();
}
```

3. Implementación del Subject (WeatherStation)

⚠ Notá que `update()` NO recibe parámetros.

El observer tendrá que "tirar" (pull) la info desde el subject.

```
java Copiar código

import java.util.ArrayList;
import java.util.List;

class WeatherStation implements Subject {
    private List<Observer> observers = new ArrayList<>();
    private float temperatura;

    @Override
    public void agregarObserver(Observer o) {
        observers.add(o);
    }

    @Override
    public void eliminarObserver(Observer o) {
        observers.remove(o);
    }

    @Override
    public void notificarObservers() {
        for (Observer o : observers) {
            o.update();
        }
    }

    // Cambia el estado
    public void setTemperatura(float nuevaTemp) {
        this.temperatura = nuevaTemp;
        notificarObservers();
    }

    // Método que los observers deben usar para "pull"
    public float getTemperatura() {
        return temperatura;
    }
}
```

4. Observer concreto (Display)

El Observer recibe la notificación, pero pide los datos al Subject.

```
java Copiar código

class Display implements Observer {
    private WeatherStation estacion; // referencia al Subject

    public Display(WeatherStation estacion) {
        this.estacion = estacion;
    }

    @Override
    public void update() {
        // Hace "pull": consulta al Subject
        float temp = estacion.getTemperatura();
        System.out.println("Display: temperatura actual = " + temp);
    }
}
```

5. Uso

```
java Copiar código

public class Main {
    public static void main(String[] args) {
        WeatherStation station = new WeatherStation();

        Observer d1 = new Display(station);
        Observer d2 = new Display(station);

        station.agregarObserver(d1);
        station.agregarObserver(d2);

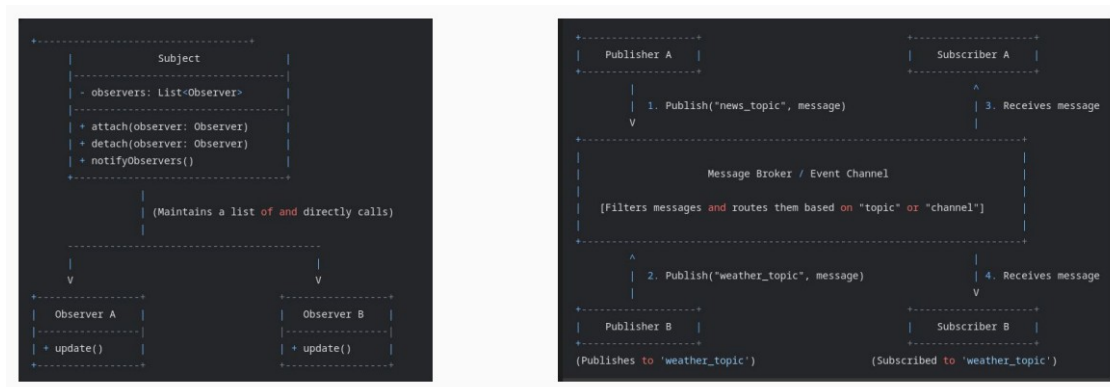
        station.setTemperatura(22.4f);
        station.setTemperatura(28.0f);
    }
}
```

Desventajas del patrón: no hay un orden de actualización garantizado (si múltiples observers dependen entre sí o del orden en que se actualizan, los resultados pueden ser inconsistentes a menos que se maneje la secuencia explícitamente) y

es más difícil de debuggear (dado que las notificaciones ocurren de forma implícita, es difícil rastrear el flujo de actualizaciones o determinar qué causó un cambio en particular).

Diferencia con Publisher - Subscriber

El Observer es un caso más sencillo de PVS. En PVS van a haber múltiples fuentes de información (los publishers), un canal donde se sube esa información, y unos subscribers que se suscriben a ese canal. Sin embargo, los Publisher no quieren mostrarle la información a todos, y los subscribers no les interesa la información de todos. Así, esa información se va filtrando en distintos tópicos o canales.



BUILDER

Es un patrón creacional cuyo objetivo es construir objetos complejos paso a paso, permitiendo separar el qué se construye con el cómo, crear objetos inmutables, evitar constructores gigantes con mil parámetros, tener distintas representaciones del mismo objeto. Es especialmente útil cuando un objeto tiene muchos parámetros opcionales, o cuando la construcción requiere varios pasos.

Esto es mucho mejor que usar setters, ya que usarlos puede generar que el objeto quede en un estado inconsistente, generando problemas en un ambiente multithreading

VS FACTORY

Mientras que Factory busca abstraer el proceso de creación de diferentes tipos de objeto (el foco es qué objeto crear), *Builder* busca resolver múltiples opciones y parámetros opcionales (el foco es cómo crear el objeto de manera flexible y paso a paso).



Factory Method

Te devuelve distintos tipos de enemigos en un juego:

java

[Copiar código](#)

```
interface Enemigo { }
class Robot implements Enemigo { }
class Zombie implements Enemigo { }

class EnemigoFactory {
    public static Enemigo crear(String tipo) {
        return switch (tipo) {
            case "robot" -> new Robot();
            case "zombie" -> new Zombie();
            default -> throw new IllegalArgumentException();
        };
    }
}
```

Uso:

java

[Copiar código](#)

```
Enemigo e = EnemigoFactory.crear("robot");
```

👉 La pregunta es: ¿qué enemigo quiero?



Builder

Constuye el mismo tipo de enemigo pero con distintas configuraciones:

java

[Copiar código](#)

```
class Robot {
    private final int vida;
    private final int velocidad;
    private final boolean blindado;

    private Robot(Builder b) {
        this.vida = b.vida;
        this.velocidad = b.velocidad;
        this.blindado = b.blindado;
    }

    static class Builder {
        private int vida = 100;
        private int velocidad = 10;
        private boolean blindado = false;

        Builder vida(int v) { this.vida = v; return this; }
        Builder velocidad(int v) { this.velocidad = v; return this; }
        Builder blindado(boolean b) { this.blindado = b; return this; }

        Robot build() { return new Robot(this); }
    }
}
```

Uso:

java

[Copiar código](#)

```
Robot r = new Robot.Builder()
    .vida(200)
    .velocidad(30)
    .blindado(true)
    .build();
```

👉 La pregunta es: ¿cómo quiero que sea el robot?

VISITOR

Patrón de comportamiento que permite agregar operaciones nuevas a una jerarquía de clases SIN modificarlas. Esto se logra separando las operaciones en objetos externos llamados *visitors*.

1. Interfaz ElementoVisitabe

```
java Copiar código

interface Figura {
    void accept(FiguraVisitor visitor);
}
```

2. Clases concretas

```
java Copiar código

class Circulo implements Figura {
    double radio;

    public Circulo(double radio) {
        this.radio = radio;
    }

    @Override
    public void accept(FiguraVisitor visitor) {
        visitor.visit(this);
    }
}

class Rectangulo implements Figura {
    double ancho, alto;

    public Rectangulo(double ancho, double alto) {
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override
    public void accept(FiguraVisitor visitor) {
        visitor.visit(this);
    }
}
```

3. Interfaz del Visitor

```
java Copiar código

interface FiguraVisitor {
    void visit(Circulo c);
    void visit(Rectangulo r);
}
```

4. Visitor concreto: calcular área

```
java Copiar código

class AreaVisitor implements FiguraVisitor {

    @Override
    public void visit(Circulo c) {
        double area = Math.PI * c.radio * c.radio;
        System.out.println("Área del círculo: " + area);
    }

    @Override
    public void visit(Rectangulo r) {
        double area = r.ancho * r.alto;
        System.out.println("Área del rectángulo: " + area);
    }
}
```

5. Uso del patrón

```
java 📄 Copiar código

public class Main {
    public static void main(String[] args) {
        List<Figura> figuras = List.of(
            new Circulo(2),
            new Rectangulo(3, 4)
        );

        FiguraVisitor areaVisitor = new AreaVisitor();

        for (Figura f : figuras) {
            f.accept(areaVisitor); // doble despacho: polimorfismo
        }
    }
}
```

Se usa cuando la jerarquía de clases está cerrada (no cambia) pero necesitas agregar múltiples operaciones nuevas y no quieres modificar las clases originales

VS STRATEGY

Strategy permite cambiar algoritmos de forma intercambiable. El objeto usa “estrategias” para ejecutar un comportamiento”. Se usa para variar el cómo hace algo (ejemplo: elegir entre distintos métodos de pago, de ordenamiento, etc.).

En cambio, visitor permite agregar nuevas operaciones a una jerarquía de clases sin modificarlas. Las clases aceptan un “visitor” (double dispatch, es decir, el método que finalmente se ejecuta depende del tipo del Visitor y del tipo del elemento visitado). Se usa para variar el qué se hace con algo (ejemplo: calcular área de figuras).

Entonces la diferencia esencial es que *Strategy* busca cambiar el comportamiento interno mediante algoritmos, y *Visitor* busca agregar operaciones externas a una estructuras de objetos.

VENTAJAS Y DESVENTAJAS

Ventajas:

- Single Responsibility Principle: separa los comportamientos de los objetos.
- Open/Closed Principle: permite agregar nuevas operaciones fácilmente, sin modificar las clases de los objetos modificados.
- Además, un *visitor* puede acumular información útil mientras trabaja con varios objetos. Esto puede ser útil cuando se quiere recorrer alguna estructura de datos compleja, como un árbol, y aplicar el *visitor* a cada objeto de esa estructura

Desventajas:

- Rompe el encapsulamiento
- Alto acoplamiento con las clases visitadas
- Agregar un nuevo tipo de objeto requiere modificar todos los *visitor*.

CODING DOJO

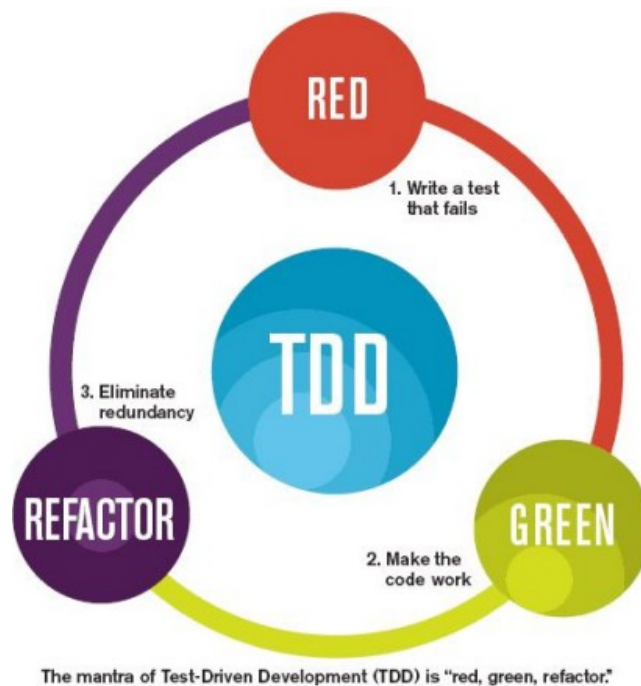
KATA

Se asocia a las artes marciales. Es un ejercicio de repetición de una serie de pasos o movimientos definidos. Su objetivo no es llegar a una respuesta correcta, sino lo que se aprende en el camino (el objetivo es la práctica, no la solución)

CODING DOJO

Un lugar seguro para prácticas y para hacer errores y aprender.

TDD



- 1) No puedes escribir código hasta que no se solucionen un test
- 2) No puedes escribir más código que el suficiente para que el test falle.
- 3) No puedes escribir más código que el suficiente para que el test fallando pase.

REFACTORING

Es una técnica para reestructurar el cuerpo existente del código, alterando su estructura interna sin cambiar el comportamiento externo. Esto lo logramos mediante tests unitarios que garanticen que el comportamiento externo no cambió, y aplicando las reestructuraciones propuestas

REFACTORING FLOW

- 1) Asegurarse que todos los tests pasan

- 2) Encontrar código que “huela” (*code smells*)
- 3) Encontrar la reestructuración
- 4) Aplicar la reestructuración

CODE SMELLS

Señal de que algo en el código no está del todo bien diseñado, aunque todavía funcione correctamente

- Duplicated Code: puede darse en una misma clase o en clases separadas. Se refiere más que nada a la lógica del código.
- Long Method
- Large Class
- Long Parameter List
- Divergent Change: se da cuando una misma clase debe modificarse en muchos lugares diferentes cada vez que se realiza un cambio en un único motivo de modificación
- Shotgun Surgery: un pequeño cambio en un requerimiento obliga a modificar muchas clases o módulos diferentes
- Feature Envy: el método “envidia” las features de otra clase, porque depende demasiado de ella
- Data clumps: cuando se ven varios parámetros, campos o variables que suelen venir en bloque y viajan juntos a todos lados, probablemente deberían ser un objeto.
- Primitive Obsession: en vez de usar objetos propios para representar conceptos más complejos, se usan los tipos de datos primitivos (int, string, float, bool, etc.)
- Switch Statements: cuando un switch aparece en varios lugares y siempre depende del mismo campo para decidir qué hacer, eso indica violación del polimorfismo y alta probabilidad de duplicación
- Lazy Class: la clase es tan pequeña o inútil que no aporta valor real, y su presencia complica innecesariamente el diseño
- Message Chains: un objeto necesita ir “atravesando” otros objetos para llegar a la información o comportamiento que realmente quiere.
- Data Class: una clase solo contiene datos (atributos) pero casi ningún comportamiento propio (básicamente solo tiene getters y setters).

TDD

En la práctica, TDD a menudo se siente como una tarea que consume mucho tiempo. Cuando se acerca un deadline, nadie tiene realmente tiempo para escribir una serie de pruebas que ni siquiera existe todavía.

Por otro lado, como sabemos, el código generado por IA no es perfecto (todavía), pero tener tests definiendo comportamiento puede ayudar mucho.

PROMPT ENGINEERING

Cuando se escribe un prompt, se intenta configurar el modelo para que prediga la secuencia correcta de tokens. Es el proceso de diseñar prompts de alta calidad que guíen a los modelos de lenguaje a producir resultados precisos. Siempre está bueno darle un contexto al prompt.

TÉCNICAS:

- **Zero-shot:** no poner ejemplos y decir qué hacer.
- **One-shot:** le doy un ejemplo y una respuesta esperada.
- **System prompting:** una pregunta que se va a ejecutar siempre y que es invisible para el usuario. Es el mensaje del "sistema" que define cómo debe comportarse el modelo, cuál es su rol, sus límites, su tono, y cómo debe interpretar las instrucciones del usuario.
- **Role prompting:** se le indica que actúe de cierta forma, que tome cierto rol, para dar una respuesta adecuada.

CONTEXT ENGINEERING

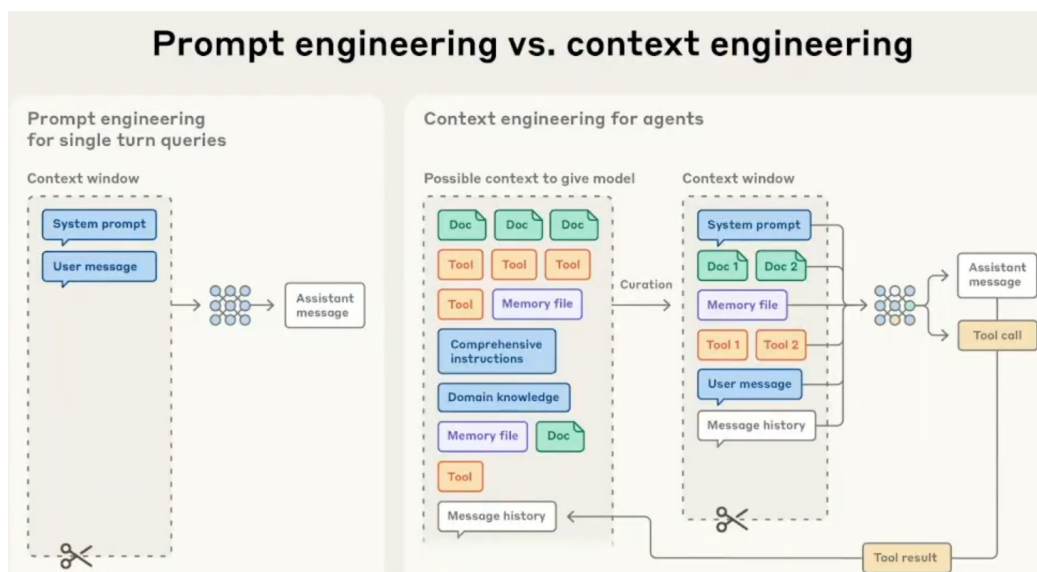
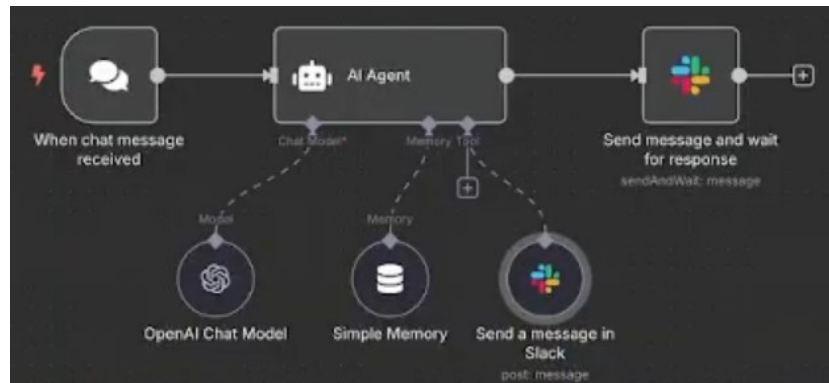
Context rot: a medida que aumenta el número de tokens en la ventana de contexto, la capacidad del modelo para recordar con precisión la información de ese contexto disminuye. En otras palabras, se empieza a ignorar lo que se dijo al principio.

Context Engineering es entonces una serie de métodos para obtener, mejorar y optimizar el conocimiento del sistema. Algunos de estos métodos son: dar a conocer las herramientas disponibles; Índex; memoria (short y long term); Prompt Engineering; compactar información.

AGENTE

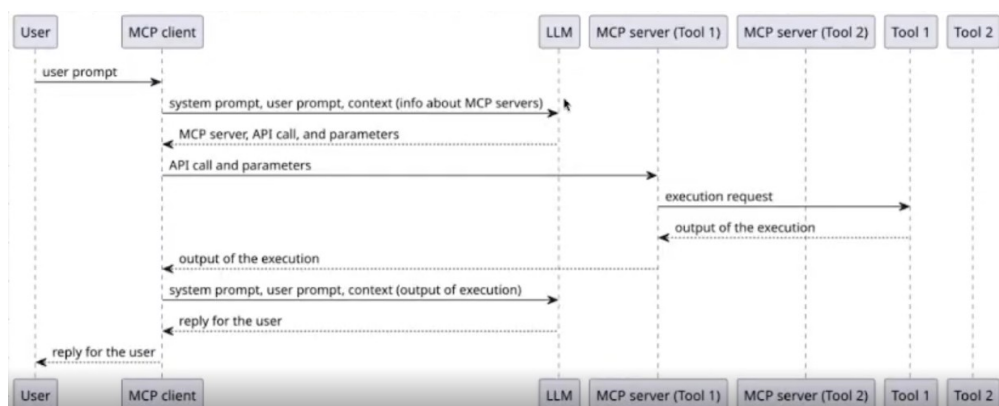
Es un sistema que usa IA para perseguir objetivos y completar tareas en nombre de los usuarios.

Vs LLM: no solo generan texto, sino que ejecutan tareas, se conectan con herramientas externas, iteran y toman decisiones. Componentes: Model, Tools, Memory, Guardrails



MCP (MODEL CONTEXT PROTOCOL)

Estándar para conectar aplicaciones de IA a sistemas externos. Son como las “nuevas APIs”.



RAG (RETRIEVAL-AUGMENTED GENERATION)

LLMs están limitados por contexto y *knowledge cutoff* (en qué momento se dejó de enviar información para entrenar al agente). RAG arregla esto permitiendo que el modelo busque información antes de contestar. Los vectores de databases permiten encontrar esta información con facilidad.

SPEC-DRIVEN DEVELOPMENT

Una especificación (spec) es un artefacto estructurado orientado al comportamiento escrito en lenguaje natural que expresa la funcionalidad del software y sirve como guía para los agentes de IA. Actualmente las especificaciones se descartan cuando comienza el “trabajo real”. Con SSD, las especificaciones (no el código) son el artefacto principal.